

Building Agents with Cloudflare Code Mode

A GitHub MCP repo assistant on Workers, Durable Objects, Workers AI, and Dynamic Workers.



WORKSHOP STARTER

github.com/thebongy/Cloudflare-Workshop-Starter

Workshop Structure

01

FIRST 30 MIN

Cloudflare primitives

Workers / Bindings / Durable Objects / Agents SDK / MCP / Code Mode / Dynamic Workers

02

NEXT 30-40 MIN

Build the agent

React chat UI / Workers AI Kimi K2.5 / GitHub MCP / Direct MCP baseline / Code Mode execution trace

» Cloudflare Workers

Workers are serverless functions deployed across Cloudflare's global network.

- 01 Workers are deployed across Cloudflare's network, not pinned to one region.
- 02 Anycast routes users to a nearby Cloudflare location for each request.
- 03 Workers run on V8 isolates, which keeps startup lightweight and avoids cold starts.

Cloudflare Workers

EXAMPLE

```
1 export default {  
2   // fetch is the Worker entrypoint.  
3   async fetch(request: Request, env: Env) {  
4     const url = new URL(request.url);  
5  
6     if (url.pathname === "/health") {  
7       return Response.json({ ok: true });  
8     }  
9  
10    // Return any standard Web Response.  
11    return new Response("Hello from Cloudflare Workers", {  
12      headers: { "content-type": "text/plain" }  
13    });  
14  }  
15 } satisfies ExportedHandler<Env>;
```

» Bindings

Bindings are the explicit capabilities the Worker is allowed to use.

- 01 Bindings let Workers access other Cloudflare primitives through a functional interface on `env`.
-

Bindings

EXAMPLE

```
1 // wrangler.jsonc
2 {
3   // Declare a Workers AI binding named AI.
4   "ai": { "binding": "AI" }
5 }
6
7 // src/index.ts
8 type Env = { AI: Ai };
9 export default {
10   async fetch(request: Request, env: Env) {
11     // AI is available through the env binding.
12     const model = "@cf/moonshotai/kimi-k2.5";
13     const message = {
14       role: "user",
15       content: "Explain AI."
16     };
17     const result = await env.AI.run(model, {
18       messages: [message]
19     });
20     return Response.json(result);
21   }
22 } satisfies ExportedHandler<Env>;
```



Okay, but what if we want to
store state/data across
requests on Workers?



Durable Objects

Durable Objects are globally addressable, stateful compute objects.

- 01 Each object has a unique ID and a single active instance that coordinates access to its state.
- 02 They are useful for sessions, coordination, counters, collaborative rooms, and strongly consistent state.
- 03 Storage lives with the object, so code and durable state are colocated behind one address.



Durable Objects

EXAMPLE

```
1 // This class owns one object's durable state.
2 export class Counter extends DurableObject<Env> {
3   async fetch() {
4     const current =
5       (await this.ctx.storage.get<number>("count")) ?? 0;
6
7     await this.ctx.storage.put("count", current + 1);
8     return Response.json({ count: current + 1 });
9   }
10 }
11
12 export default {
13   async fetch(request: Request, env: Env) {
14     // Different names create different objects.
15     const idA = env.COUNTERS.idFromName("repo:a");
16     const idB = env.COUNTERS.idFromName("repo:b");
17
18     const a = env.COUNTERS.get(idA);
19     const b = env.COUNTERS.get(idB);
20
21     // The same pattern scales to millions of IDs.
22     const responseA = await a.fetch(request);
23     const responseB = await b.fetch(request);
24
25     const countA = await responseA.json();
26     const countB = await responseB.json();
27
28     return Response.json({ countA, countB });
29   }
30 }
```



Turns out, Durable Objects are perfect primitives for AI agents: stateful code execution. So we made an SDK to make it even easier.



Agents SDK

Agents SDK provides the agent-shaped runtime on top of Workers and Durable Objects.

- 01 It adds state, routing, WebSocket connections, and lifecycle hooks for long-running agents.
- 02 AIChatAgent provides a chat-oriented base class that persists conversation state.
- 03 Agents can coordinate model calls, tools, schedules, and external connections from one stateful object.



Agents SDK

EXAMPLE

```
1 import { AIChatAgent } from "@cloudflare/ai-chat";
2 import { streamText, convertToModelMessages } from "ai";
3 import { createWorkersAI } from "workers-ai-provider";
4
5 // AIChatAgent gives this class a persistent chat session.
6 export class ChatAgent extends AIChatAgent<Env> {
7   async onChatMessage() {
8     const workersai =
9       createWorkersAI({ binding: this.env.AI });
10
11     // Convert stored UI messages for the model.
12     const messages =
13       await convertToModelMessages(this.messages);
14
15     const result = streamText({
16       model: workersai("@cf/moonshotai/kimi-k2.5"),
17       messages
18     });
19
20     // Return a stream the chat UI can consume.
21     return result.toUIMessageStreamResponse();
22   }
23 }
```



MCP is a standard protocol for connecting models to external tools and data.

- 01 It supplies tool discovery, tool descriptions, schemas, and auth integration.
- 02 A model client can connect to many MCP servers without hard-coding every integration.
- 03 The same interface can expose local files, SaaS APIs, databases, or developer tools.



MCP

EXAMPLE

```
1 // Add GitHub as an MCP server over HTTP.
2 await this.addMcpServer(
3   "github",
4   "https://api.githubcopilot.com/mcp/",
5   {
6     transport: {
7       type: "streamable-http",
8       headers: {
9         Authorization: "Bearer <GITHUB_MCP_PAT>",
10        // Keep demo access read-only.
11        "X-MCP-Readonly": "true"
12      }
13    }
14  }
15 );
16
17 // Expose discovered MCP tools to the model.
18 const tools = this.mcp.getAITools({
19   serverName: "github"
20 });
```

The Problem With Tool Calling

Direct tool calling can make multi-step workflows expand into many model turns and lots of context.

- 01 Each tool result is added back into the conversation, even when it is only intermediate data.
- 02 The model has to choose the next call after every result instead of keeping local state in one workflow.
- 03 Example: stale PRs means list PRs, inspect checks, inspect timelines, rank owners, then summarize.



What if we make the model
generate code to call tools
programmatically instead?

Code Mode

Code Mode turns MCP tools into a typed JavaScript API the model can program against.

- 01 The model writes one small program instead of many separate tool calls.
- 02 Loops, filters, local variables, and Promise.all happen inside the program.
- 03 The host can inspect the generated code, tool calls, logs, result, and errors.

EXECUTION FLOW



Code Mode

HOST WORKER

```
1  const githubTools = this.mcp.getAITools({
2    serverName: "github"
3  });
4
5  // Expose GitHub MCP as code.
6  const codemode = createCodeTool({
7    tools: githubTools,
8    executor
9  });
10
11  await generateText({
12    model,
13    tools: { codemode },
14    // Model writes the workflow.
15    toolChoice: {
16      type: "tool",
17      toolName: "codemode"
18    },
19    prompt: "Find stale PRs with failing checks."
20  });
```

GENERATED CODE

```
1  const prs = await mcp.listPullRequests({
2    owner, repo, state: "open"
3  });
4
5  // Generated code keeps local state.
6  const stale = [];
7  for (const pr of prs) {
8    const checks = await mcp.listCheckRuns({
9      owner, repo, ref: pr.head.sha
10   });
11
12   // Filter before returning to the model.
13   if (checks.some((c) => c.conclusion === "failure")) {
14     stale.push({
15       number: pr.number,
16       title: pr.title,
17       failures: checks.length
18     });
19   }
20 }
21
22 return stale.slice(0, 5);
```



But where do we run the
code generated by the
agent?



Dynamic Workers

Dynamic Workers run dynamic code in isolated Worker environments.

- 01 They are separate Worker isolates, not plain eval inside the host Worker.
- 02 The host controls which capabilities are exposed to the dynamic runtime.
- 03 Outbound network access can be disabled or constrained for generated code.



Dynamic Workers

EXAMPLE

```
1 // Create an isolated runtime for generated code.
2 const executor = new DynamicWorkerExecutor({
3   loader: env.LOADER,
4   // Disable global outbound fetch.
5   globalOutbound: null,
6   timeout: 30_000
7 });
8
9 const output = await executor.execute(code, [
10   resolveProvider({ tools: githubTools })
11 ]);
12
13 // Return inspectable execution details.
14 return {
15   result: output.result,
16   logs: output.logs,
17   error: output.error
18 };
```

Let's build a GitHub agent on Cloudflare using Code Mode!



WORKSHOP STARTER

github.com/thebongy/Cloudflare-Workshop-Starter